Session 4 — Solutions

Perceptrons and Multi-Layer Networks | Neural Networks for Engineers

Exercise 1 : Identifying Perceptrons that Implement the AND Gate

Question 1 — Output table

For each perceptron, we compute $z = w_1x_1 + w_2x_2 + b$ then apply $f(z)$.

Perceptron A ($w_1 = 1,\ w_2 = 1,\ b = -2$):

x_1	x_2	$z = x_1 + x_2 - 2$	$\hat{y} = f(z)$
0	0	-2	0
0	1	-1	0
1	0	-1	0
1	1	0	**1**

Perceptron B ($w_1 = 1,\ w_2 = 1,\ b = -1$):

x_1	x_2	$z = x_1 + x_2 - 1$	$\hat{y} = f(z)$
0	0	-1	0
0	1	0	**1**
1	0	0	**1**
1	1	1	**1**

Perceptron C ($w_1 = 0.5,\ w_2 = 0.5,\ b = -0.8$):

x_1	x_2	$z = 0.5x_1 + 0.5x_2 - 0.8$	$\hat{y} = f(z)$
0	0	-0.8	0
0	1	-0.3	0
1	0	-0.3	0
1	1	0.2	**1**

Perceptron D ($w_1 = 2,\ w_2 = 2,\ b = -3$):

x_1	x_2	$z = 2x_1 + 2x_2 - 3$	$\hat{y} = f(z)$
0	0	-3	0
0	1	-1	0
1	0	-1	0
1	1	1	**1**

Question 2 — Which perceptrons implement AND?

Perceptrons **A**, **C**, and **D** correctly implement the AND gate. Perceptron **B** fails.

Question 3 — Why does perceptron B fail?

Perceptron B uses $b = -1$, so the threshold $z \geq 0$ is reached as soon as $x_1 + x_2 \geq 1$, i.e. whenever *at least one* input is 1. This implements the **OR** gate, not AND. The bias is too large (too close to zero), so the boundary line $x_1 + x_2 = 1$ passes below the point $(1, 1)$ but also excludes $(0, 1)$ and $(1, 0)$ on the wrong side.

Question 4 — Uniqueness of the solution

There are **infinitely many** perceptrons implementing AND.
Geometrically, the decision boundary is a line $w_1x_1 + w_2x_2 + b = 0$. The only constraint is that this line must strictly separate the point $(1, 1)$ (class 1) from the three other points $(0, 0)$, $(0, 1)$, $(1, 0)$ (class 0). Any line passing through the open strip between these two groups is a valid solution. Scaling all weights and the bias by a positive constant gives another valid perceptron, and any line within the feasible region also works — hence there is a continuum of solutions.

Exercise 2 : Decision Boundary of a Single Perceptron

Question 1 — Decision boundary equation

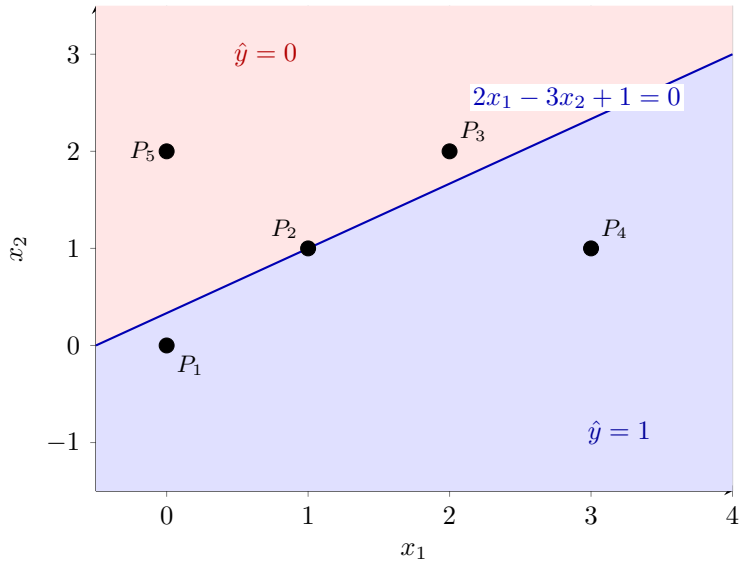
The boundary is the set of points where $\hat{y}$ switches, i.e. where $z = 0$:

$$2x_1 - 3x_2 + 1 = 0 \implies x_2 = \frac{2}{3}x_1 + \frac{1}{3}.$$

$$x_2 = \frac{2}{3}x_1 + \frac{1}{3} \qquad (\text{slope } \alpha = \tfrac{2}{3}, \text{ intercept } \beta = \tfrac{1}{3}).$$

Question 2 — Plot of the boundary

The boundary passes through $(0, \frac{1}{3})$ and $(1, 1)$. The region $\hat{y} = 1$ corresponds to $2x_1 - 3x_2 + 1 \geq 0$, i.e. *below* the line.



Question 3 — Classification of points

Point	x_1	x_2	$z = 2x_1 - 3x_2 + 1$	$\hat{y}$
P_1	0	0	1	1
P_2	1	1	0	1
P_3	2	2	-1	0
P_4	3	1	4	1
P_5	0	2	-5	0

Question 4 — Adjusting the bias for $Q = (1, 2)$

First, check the current classification of Q:

$$z_Q = 2(1) - 3(2) + 1 = 2 - 6 + 1 = -3 < 0 \;\Rightarrow\; \hat{y} = 0.$$

Q is currently in class 0. We want $\hat{y} = 1$, i.e. $z_Q \geq 0$:

$$2(1) - 3(2) + b \geq 0 \;\Rightarrow\; -4 + b \geq 0 \;\Rightarrow\; b \geq 4.$$

Any bias $b \geq 4$ classifies $Q = (1, 2)$ as class 1 while keeping $w_1 = 2$, $w_2 = -3$ unchanged. The minimal change is $b = 4$ (the boundary passes exactly through Q).
Note: increasing b shifts the boundary line upward ($x_2 = \frac{2}{3}x_1 + \frac{b}{3}$), enlarging the region classified as 1.

Exercise 3 : Forward Propagation in a 2-2-1 MLP

Question 1 — Complete forward pass

We compute $z_i^{(1)} = \sum_j W_{ij}^{(1)} x_j + b_i^{(1)}$ then $h_i = f(z_i^{(1)})$, then $z^{(2)} = W^{(2)\top} h + b^{(2)}$ and $\hat{y} = f(z^{(2)})$.

Input $(0,0)$**:**

$$z_1^{(1)} = 1 \cdot 0 + 1 \cdot 0 - 1.5 = -1.5 \;\Rightarrow\; h_1 = 0$$
$$z_2^{(1)} = 1 \cdot 0 + (-1) \cdot 0 - 0.5 = -0.5 \;\Rightarrow\; h_2 = 0$$
$$z^{(2)} = 1 \cdot 0 + 1 \cdot 0 - 1.5 = -1.5 \;\Rightarrow\; \hat{y} = 0$$

Input $(0,1)$**:**

$$z_1^{(1)} = 0 + 1 - 1.5 = -0.5 \;\Rightarrow\; h_1 = 0$$
$$z_2^{(1)} = 0 - 1 - 0.5 = -1.5 \;\Rightarrow\; h_2 = 0$$
$$z^{(2)} = 0 + 0 - 1.5 = -1.5 \;\Rightarrow\; \hat{y} = 0$$

Input $(1,0)$**:**

$$z_1^{(1)} = 1 + 0 - 1.5 = -0.5 \;\Rightarrow\; h_1 = 0$$
$$z_2^{(1)} = 1 + 0 - 0.5 = 0.5 \;\Rightarrow\; h_2 = 1$$
$$z^{(2)} = 0 + 1 - 1.5 = -0.5 \;\Rightarrow\; \hat{y} = 0$$

Input $(1,1)$**:**

$$z_1^{(1)} = 1 + 1 - 1.5 = 0.5 \;\Rightarrow\; h_1 = 1$$
$$z_2^{(1)} = 1 - 1 - 0.5 = -0.5 \;\Rightarrow\; h_2 = 0$$
$$z^{(2)} = 1 + 0 - 1.5 = -0.5 \;\Rightarrow\; \hat{y} = 0$$

Summary table:

x_1	x_2	$z_1^{(1)}$	h_1	$z_2^{(1)}$	h_2	$z^{(2)}$	$\hat{y}$
0	0	-1.5	0	-0.5	0	-1.5	**0**
0	1	-0.5	0	-1.5	0	-1.5	**0**
1	0	-0.5	0	0.5	1	-0.5	**0**
1	1	0.5	1	-0.5	0	-0.5	**0**

Question 2 — Logical function

The output is always $\hat{y} = 0$: this network does **not** implement any useful logical function as configured. It is equivalent to the *constant zero* function.

Why? The output neuron fires only if $h_1 + h_2 \geq 1.5$, i.e. only when *both* $h_1 = 1$ *and* $h_2 = 1$ simultaneously. But $h_1 = 1$ requires $x_1 + x_2 \geq 1.5$ (both inputs at least partially active), while $h_2 = 1$ requires $x_1 - x_2 \geq 0.5$ (i.e. x_1 strictly larger than x_2). No binary input pair satisfies both conditions at once.

Remark for students: this illustrates how weight choice is critical. Because $W^{(2)} = (1,\, 1)^\top$, the output neuron cannot distinguish which hidden neuron fired: it sees only $h_1 + h_2$. Since h_1 and h_2 never fire simultaneously on binary inputs (as shown above), $h_1 + h_2 \in \{0, 1\}$ for all inputs, and no value of $b^{(2)}$ alone can separate $(1, 1)$ from $(1, 0)$.
To implement AND, one must also change the output weights. For instance, setting $W^{(2)} = (1,\, 0)^\top$ and $b^{(2)} = -0.5$ gives $\hat{y} = f(h_1 - 0.5)$, which fires exactly when $h_1 = 1$, i.e. only for the input $(1, 1)$ — correctly implementing AND.
Alternatively, setting $W^{(1)}_{2,\cdot} = (1, 1)$ with $b^{(1)}_2 = -0.5$ (so h_2 becomes a copy of h_1 shifted by a different threshold) and keeping $W^{(2)} = (1, 1)^\top$ with $b^{(2)} = -1.5$ would also work, since both h_1 and h_2 would then fire together.

Question 3 — Geometric interpretation of hidden neurons

Hidden neuron h_1 fires when $z^{(1)}_1 \geq 0$:

$$x_1 + x_2 - 1.5 \geq 0 \iff x_1 + x_2 \geq 1.5.$$

This is the half-plane *above* the diagonal line $x_1 + x_2 = 1.5$. It only activates when both inputs are simultaneously large.

Hidden neuron h_2 fires when $z^{(1)}_2 \geq 0$:

$$x_1 - x_2 - 0.5 \geq 0 \iff x_1 - x_2 \geq 0.5.$$

This is the half-plane *below* the line $x_2 = x_1 - 0.5$, i.e. where x_1 is significantly larger than x_2.

Each hidden neuron defines a **linear boundary** in the input plane:

$$h_1: \quad x_1 + x_2 = 1.5 \qquad\qquad h_2: \quad x_1 - x_2 = 0.5$$

These two lines partition the plane into regions. The output layer then combines these two binary detectors — illustrating how an MLP builds complex regions by intersecting the half-planes of its hidden neurons.

Exercise 4 : Designing MLP Architectures

Task A — Spam Detection

a) Proposed architecture: $20 \to 16 \to 1$

- Input layer: 20 neurons (one per feature).
- Hidden layer: 16 neurons with ReLU activation.

- Output layer: 1 neuron with **sigmoid** activation (binary probability).

b) Parameter count:

$$\underbrace{(20 \times 16) + 16}_{\text{input}\to\text{hidden}} + \underbrace{(16 \times 1) + 1}_{\text{hidden}\to\text{output}} = 320 + 16 + 16 + 1 = \mathbf{353} \text{ parameters.}$$

Any reasonable hidden layer size (e.g. 8, 16, 32 neurons) is acceptable. The key constraints are: **20 input neurons**, **1 output neuron with sigmoid**. The sigmoid maps the output to $[0, 1]$, interpreted as $P(\text{spam})$.

Task B — Handwritten Digit Recognition

a) Number of inputs:

$$28 \times 28 = \mathbf{784} \text{ input neurons (one per pixel).}$$

b) Output layer: 10 neurons (one per digit class, 0–9) with **softmax** activation, so that outputs sum to 1 and can be interpreted as class probabilities.

c) Architecture $784 \to 128 \to 10$ **— parameter count:**

$$\underbrace{(784 \times 128) + 128}_{\text{input}\to\text{hidden}} + \underbrace{(128 \times 10) + 10}_{\text{hidden}\to\text{output}} = 100352 + 128 + 1280 + 10 = \mathbf{101\,770} \text{ parameters.}$$

Task C — Open Question

a) Parameter count for $784 \to 10 \to 10$**:**

$$\underbrace{(784 \times 10) + 10}_{\text{input}\to\text{hidden}} + \underbrace{(10 \times 10) + 10}_{\text{hidden}\to\text{output}} = 7840 + 10 + 100 + 10 = \mathbf{7\,960} \text{ parameters.}$$

b) Critical analysis:

This architecture is **almost certainly too small** for handwritten digit recognition.
The hidden layer has only 10 neurons, creating an *information bottleneck*: the network must compress the 784-dimensional input into just 10 intermediate values before making a decision. In practice, networks of this size learn poorly because they lack the capacity to represent the variety of handwriting styles.
For reference, a single hidden layer of 128 neurons (Task B) already reaches ∼97% accuracy on MNIST with standard training. The student's architecture has ∼13× fewer parameters in the hidden layer, which strongly limits expressiveness. This illustrates that **model capacity must match task complexity**: too few neurons leads to underfitting.